
PROGETTO
2024/2025

CONTATTI

- ▶ Non esitate a contattarci per dubbi, problemi o domande

- ▶ Scriveteci a:

Bianca Raimondi: bianca.raimondi3@unibo.it

<https://www.unibo.it/sitoweb/bianca.raimondi3>

Alessandro Zanni: alessandro.zanni3@unibo.it

<https://www.unibo.it/sitoweb/alessandro.zanni3>

- ▶ Per il ricevimento mandate una mail

IL PROGETTO SERVE PER

- ▶ Farvi sperimentare le tecniche viste a lezione su scala più grande, in direzione di quello che succede nel mondo reale
- ▶ I progetti reali sono spesso MOLTO più grandi anche di questo progetto
- ▶ Mettervi alla prova in uno scenario che lascia spazio alla vostra fantasia, ma pone anche alcuni vincoli

VALUTAZIONE DEL PROGETTO

- ▶ Il progetto è parte integrante dell'esame ed è **obbligatorio**
- ▶ Ha un voto massimo di 8 punti che si sommano al voto dello scritto (max 24)
- ▶ I punti diventano al massimo 6 se si consegna il progetto dopo l'appello di Settembre 2024
- ▶ Il lavoro viene svolto in gruppi, ma la valutazione del progetto è **individuale**
 - ▶ Studenti dello stesso gruppo possono prendere voti diversi!
- ▶ Il progetto **si consegna una volta sola** ed il voto è valido per l'intero anno accademico (anche per quello successivo se le regole non cambiano)
- ▶ Chi non ottiene almeno 2 punti alla discussione orale dovrà presentare un nuovo progetto (che vi sarà dato in seguito)

GRUPPI

- ▶ Il progetto si svolge in gruppi, ma il voto **non** è di gruppo
- ▶ Il gruppo ideale è composto da 3 persone
- ▶ NON è possibile fare gruppi di 5 o più persone
- ▶ Auto-organizzatevi per la creazione di gruppi
- ▶ Comunicarete i componenti del gruppo in fase di prenotazione e/o discussione del progetto

QUANDO CONSEGNARE IL PROGETTO?

- ▶ Il progetto potrà essere presentato entro il 28/02/**2026**
- ▶ Ci saranno 4 appelli (uno/due per sessione): solitamente 1 Febbraio, 2 Giugno-Luglio, 1 Settembre
- ▶ Le date degli appelli verranno comunicate tramite Virtuale

CONSEGNA E DISCUSSIONE

- ▶ Il progetto si consegna **una settimana** prima della discussione
- ▶ **TUTTI** i membri del gruppo devono partecipare alla discussione
- ▶ Il gruppo consegna:
 - ▶ Codice sorgente
 - ▶ File README
 - ▶ Screen recording che mostri l'esecuzione del gioco
 - ▶ Breve relazione (3/4 pagine) in cui si descrivono le principali scelte nell'implementazione del progetto
- ▶ Consegna da parte di **un solo membro** del gruppo tramite Virtuale
- ▶ Aggiungete alla consegna un commento con tutte le mail dei membri del gruppo

CONTENUTI DEL GIOCO

- ▶ Si richiede di implementare una versione ASCII semplificata di Snake
- ▶ Con punteggio
- ▶ Il gioco si controlla da tastiera



SVOLGIMENTO DEL GIOCO

- ▶ Le regole del gioco seguono quelle di Snake classico: il giocatore controlla un serpente che si muove nella griglia
- ▶ Spostamento orizzontale/verticale: il giocatore può muovere il serpente fino a toccare il bordo della griglia, permettendo al serpente di spostarsi dal lato opposto
- ▶ Punteggio: il punteggio aumenta ogni volta che il serpente mangia una mela a seconda del livello
- ▶ Il gioco deve quindi prevedere un **menù** iniziale con le seguenti opzioni:
 - Nuova partita
 - Visualizza classifica ordinata in modo decrescente (punteggi delle partite terminate)

SVOLGIMENTO DEL GIOCO

- ▶ Non esistono traguardi ma ci sono livelli di gioco da gestire:
 - Al completamento del livello, si ottiene un bonus (maggiore il livello, maggiore il bonus)
 - I livelli hanno un tempo prefissato. Esempio: entro 5 min. si cerca di ottenere il punteggio maggiore
 - La lunghezza del serpente rimane uguale anche quando mangia.
 - Aumentando il livello cambia solo la velocità con cui si sposta il serpente
- ▶ GameOver: la partita termina se il serpente si morde la coda, se il tempo termina o se esce dal gioco (es. pulsante della tastiera per mettere in pausa il gioco). Al termine della partita (game over) il giocatore torna al menù iniziale.

REQUISITI

- ▶ Per il progetto è **possibile utilizzare SOLO le librerie grafiche curses/ncurses.h, ctime.h e le librerie viste a lezione.**
- ▶ La griglia deve essere realizzata con grafica ASCII e deve essere una matrice con dimensioni almeno 20x10.
- ▶ I Livelli sono implementati tramite **liste bidirezionali** che permettono di passare da un livello all'altro in qualsiasi momento, anche se il livello non è completo
- ▶ Snake è realizzato tramite matrice booleana. Il suo movimento è realizzato togliendo un pezzo dalla coda e aggiungendolo in testa

ESEMPIO DIVISIONE DEL LAVORO ALL'INTERNO DI UN GRUPPO

- ▶ Componente #1: si occupa del serpente e del suo movimento
- ▶ Componente #2: si occupa dei livelli e della loro gestione
- ▶ Componente #3: si occupa della griglia di gioco e delle collisioni (con mele e bordi della griglia).
- ▶ Altri task da suddividere:
 - ▶ Punteggio
 - ▶ Gestione menù
 - ▶ Salvataggio file

IMPOSTAZIONE DEL PROGETTO

- ▶ Il progetto deve essere realizzato usando le classi
- ▶ La classifica viene salvata in un file esterno
- ▶ Il progetto è organizzato in più file (no Main con tutto dentro)
- ▶ Ad ogni classe corrispondono due file:
 - ▶ *NomeClasse.cpp*
 - ▶ *NomeClasse.hpp*

ESEMPIO DI FILE

NomeClasse.hpp

```
class NomeClasse{  
protected:  
    int field;  
    ...  
public:  
    ...  
    void method();  
    ...  
};
```

NomeClasse.cpp

```
#include "NomeClasse.hpp"  
  
void NomeClasse::method(){  
    // do something  
}
```

IN OGNI FILE IN CUI SI USA IL TIPO "NOMECLASSE" BISOGNA IMPORTARE NOMECLASSE.HPP

LETTURA DA FILE

- ▶ Per leggere e scrivere su file bisogna includere `fstream`
- ▶ `ifstream` è un **input file stream**: legge dati da un file

```
#include <iostream>
#include <fstream>
using namespace std;
```

```
int main() {
    ifstream inputFile; /* Dichiarazione di tipo */
    inputFile.open("file.txt");
    char ch;
    /* lettura dati */
    while(!inputFile.eof()){
        inputFile.get(ch);
        cout << ch << endl;
    }
    inputFile.close();
    return 0;
}
```

- ▶ **while**(`!inputFile.eof()`) identifica un ciclo che finisce quando il file termina
- ▶ In questo contesto si tratta di un file testuale
- ▶ Tramite la funzione **get** si memorizza il carattere corrente del file nella variabile `ch`
- ▶ Alla fine del ciclo dobbiamo chiudere lo stream, ovvero il flusso di dati, proveniente dal file in ingresso, tramite l'istruzione **inputFile.close()**;

SCRITTURA SU FILE

- ▶ Per leggere e scrivere su file bisogna includere `fstream`
- ▶ `ofstream` è un **output file stream**: scrive dati su un file

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {

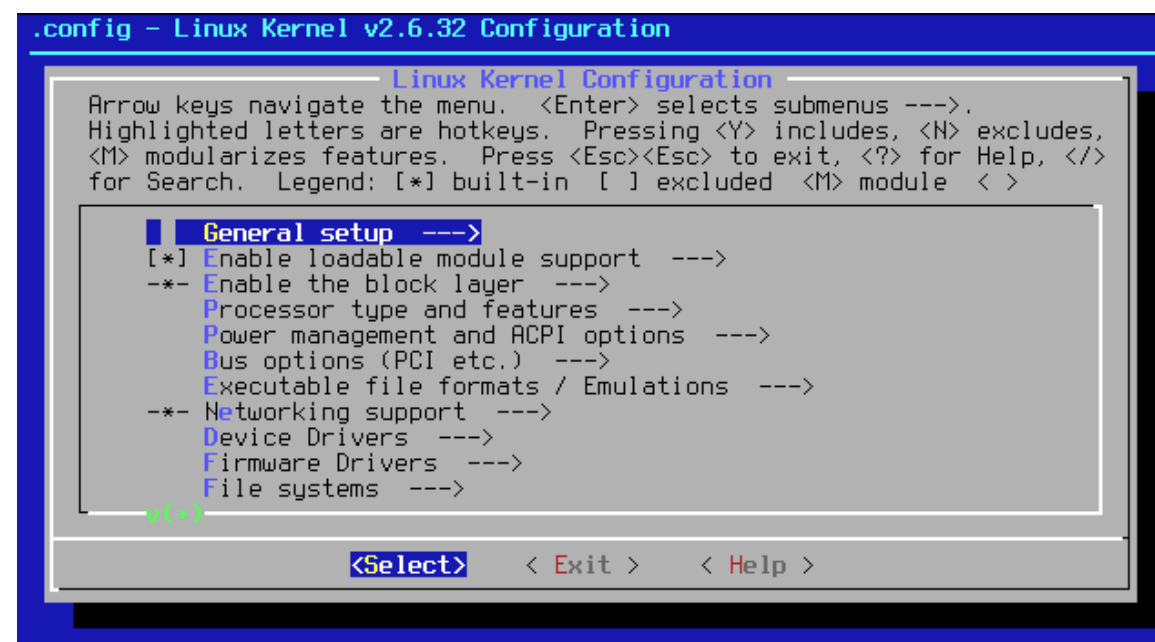
    ofstream outputFile; /* Dichiarazione di tipo */
    outputFile.open("file2.txt"); /* Apertura del file */
    outputFile << "Prova di scrittura su file";
    outputFile.close();

    return 0;
}
```

- ▶ Se il file da aprire in scrittura non è presente ne verrà creato uno vuoto con il nome specificato
- ▶ Possiamo stampare una frase su **outputFile** semplicemente ricalcando la sintassi del comando **cout**
- ▶ Per scrivere sul file senza eliminarne il contenuto già presente, è sufficiente aprire il file con il comando:
outputFile.open("file2.txt",ios::app);

LIBRERIA CURSES/NCURSES

- ▶ **ncurses** (new curses) è una libreria utilizzata per consentire al programmatore di scrivere TUI (Text-based User Interfaces).
- ▶ È un insieme di strumenti per lo sviluppo di applicazioni "GUI-like" che vengono eseguite tramite terminale.
- ▶ ncurses è un'emulazione di software libero delle **curses**
 - ▶ Su windows ncurses si chiama curses, fate attenzione in fase di import (`#include <curses.h>`)

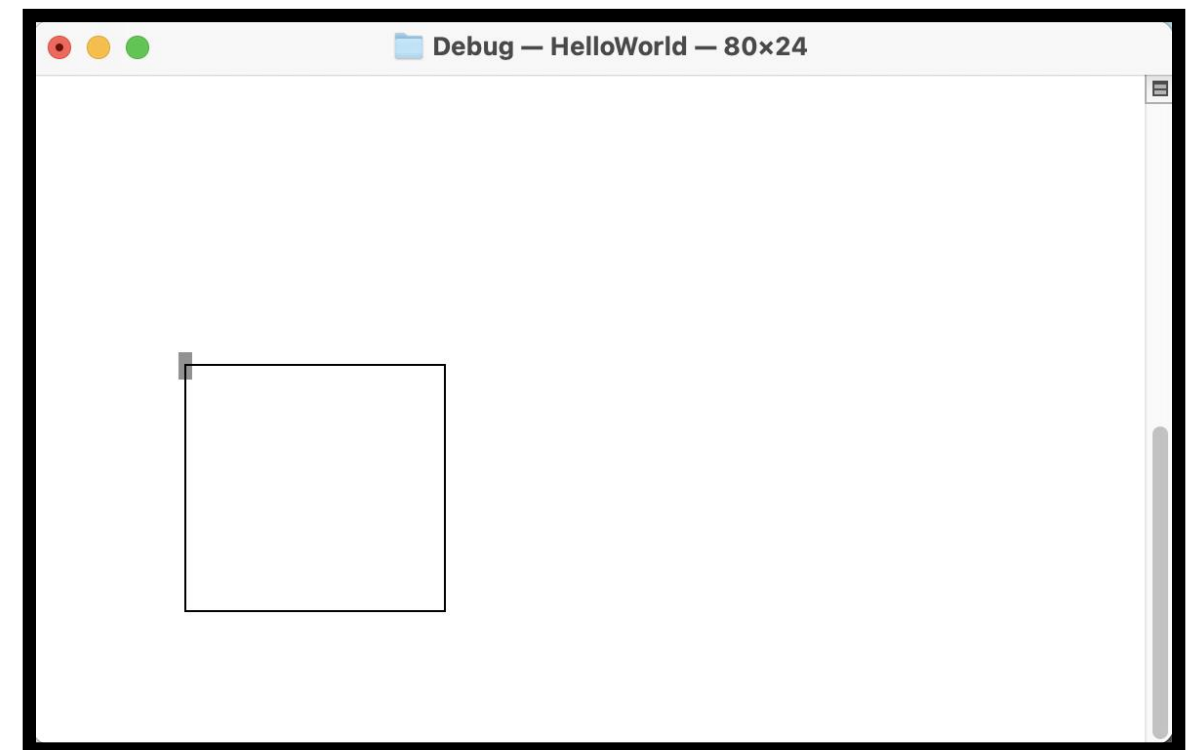


Esempio di applicazione creata con curses

LIBRERIA CURSES/NCURSES

► Esempio dell'utilizzo della libreria

```
HelloWorld.cpp X
1  #include <ncurses.h>
2  #include <iostream>
3
4  using namespace std;
5
6  int main(int argc, char ** argv){
7      //initializes the screen
8      initscr();
9
10     int height, width, start_y, start_x;
11
12     height = 10;
13     width = 20;
14     start_y = 10;
15     start_x = 10;
16
17     WINDOW * win = newwin(height, width, start_y, start_x);
18
19     //refreshes the screen
20     refresh();
21
22     //show the window
23     box(win, 0, 0);
24     wrefresh(win);
25
26     //wait for user input like "cin >> a;"
27     getch();
28
29     //deallocates memory and ends ncurses
30     endwin();
31 }
```



► Qui potete trovare una guida per l'utilizzo della libreria:
https://www.youtube.com/playlist?list=PL2U2TQ__OrQ8jTf0_noNKtHMuYlyxQl4v

LIBRERIA CURSES/NCURSES

Funzioni principali:

- ▶ **initscr()** : inizializza screen
- ▶ **endwin()** : dealloca memoria e chiude il programma
- ▶ **newwin(height, width, start_y, start_x)** : crea una nuova finestra
- ▶ **refresh()** : refresh dello screen con le informazioni aggiuntive
- ▶ **move(x, y)** : muove il cursore nella posizione indicata (x, y)
- ▶ **clear()** : cancella tutto ciò che è presente sullo screen

LIBRERIA CURSES/NCURSES - LINUX

- ▶ Installazione libreria:

sudo apt install libncurses-dev

- ▶ Compilazione:

g++ HelloWorld.cpp -lncurses -o HelloWorld

- ▶ Esecuzione:

./HelloWorld

LIBRERIA CURSES/NCURSES - MACOS

- ▶ Installazione libreria:

brew install ncurses

- ▶ Compilazione:

g++ HelloWorld.cpp -lncurses -o HelloWorld

- ▶ Esecuzione:

./HelloWorld

LIBRERIA CURSES/NCURSES - WINDOWS

- ▶ Installate una suite di tool di compilazione per Windows, ad esempio MinGW
- ▶ Scaricare il file **MinGW-w64-install.exe** da <https://sourceforge.net/projects/mingw/>
- ▶ Fate partire l'installer e andate avanti
- ▶ Selezionare le opzioni "A Basic MinGW installation" e "The GNU C++ compiler"
- ▶ Selezionare "Apply Changes" dal menu Installation
- ▶ Aggiungete al PATH di Windows il percorso: C:\MinGW\bin\

LIBRERIA CURSES/NCURSES - WINDOWS

- ▶ Da MinGW selezionare i seguenti pacchetti:

pd curses

libncurses

libpd curses

ncurses

- ▶ Compilazione:

g++ -I/mingw64/include/ncurses -o HelloWorld HelloWorld.cpp -lncurses -L/mingw64/bin -static

- ▶ Esecuzione:

HelloWorld

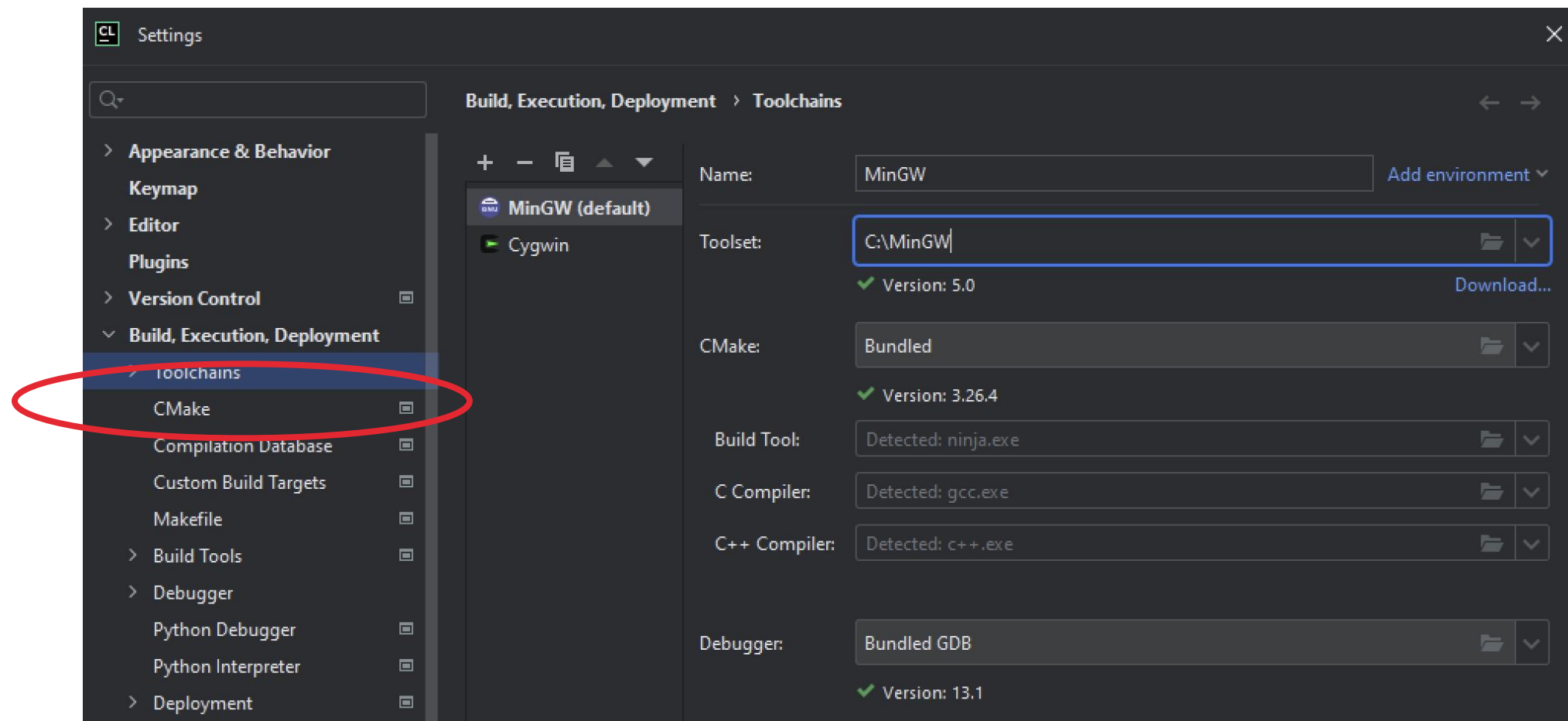
oppure

./HelloWorld

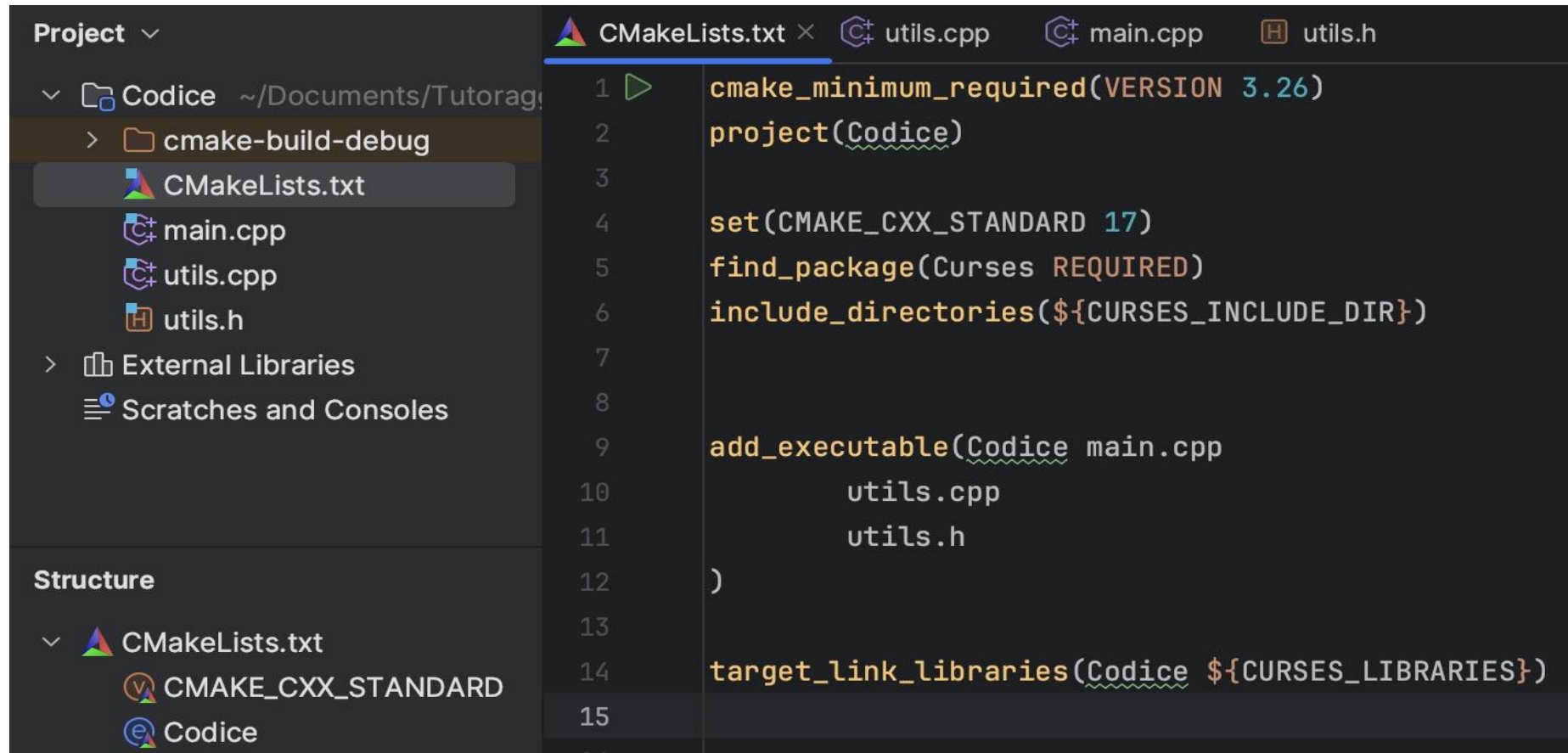
Nel caso non funzionassero i comandi è possibile eseguire il programma facendo doppio click sul file

LIBRERIA CURSES/NCURSES - PER WINDOWS

- ▶ Assicuratevi di aver selezionato il toolchain giusto per la compilazione
- ▶ Questa opzione in Clion si trova in
 - ▶ File -> Settings -> Build, Execution, Deployment -> Toolchains
 - ▶ Selezionare MinGW (default) e assicurarsi di non avere "Bundled MinGW" nel campo "Toolset"
 - ▶ In caso, dovrete modificarlo andando a scegliere la vostra installazione di MinGW (il percorso di default è C:\MinGW)



LIBRERIA CURSES/NCURSES - COMPILAZIONE CON CLION



- ▶ Aprire il file CMakeList.txt e aggiungere le seguenti linee evidenziate in grassetto:
find_package(Curses REQUIRED)
include_directories(\${CURSES_INCLUDE_DIR})
add_executable(PROJECT_NAME main.cpp)
target_link_libraries(PROJECT_NAME **\${CURSES_LIBRARIES}**)
- ▶ NB: PROJECT_NAME deve essere sostituito con il nome del vostro progetto CLION.
- ▶ Clion creerà l'eseguibile **all'interno della cartella cmake-build-debug**
- ▶ Eseguite poi il programma **utilizzando il terminale**

ULTERIORI INFORMAZIONI

- ▶ Per dubbi o domande consultate:
 - **Forum** su Virtuale (**FAQ**)
- ▶ **Prima di chiedere un ricevimento** assicuratevi che il vostro problema non sia già stato discusso o risolto da altri sul Forum
- ▶ Se siete alla ricerca di componenti del gruppo utilizzate la **chat** di Virtuale